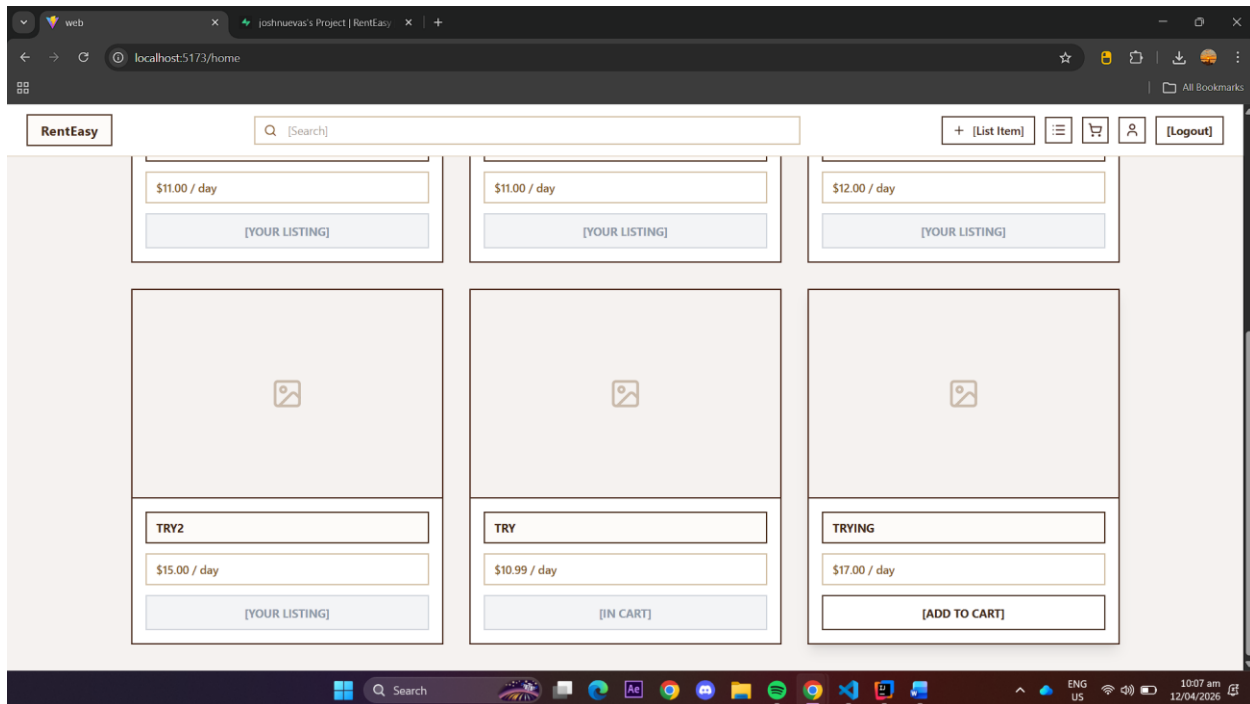
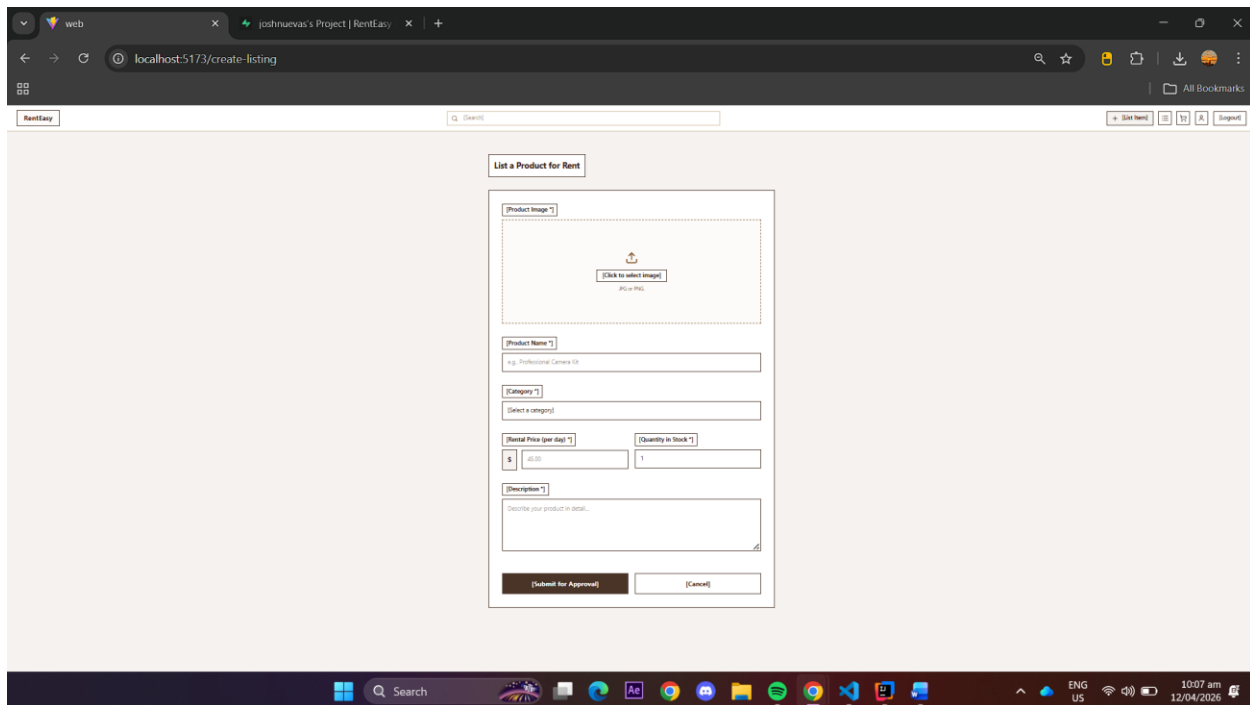


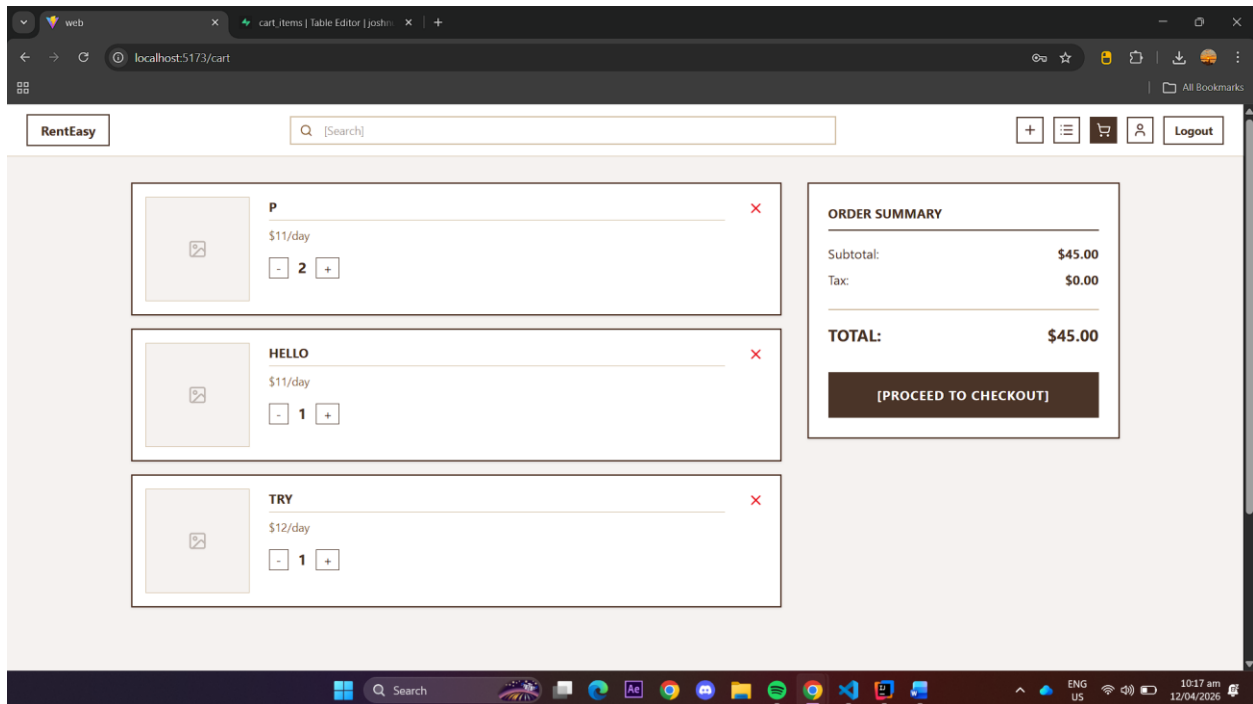
Home Page



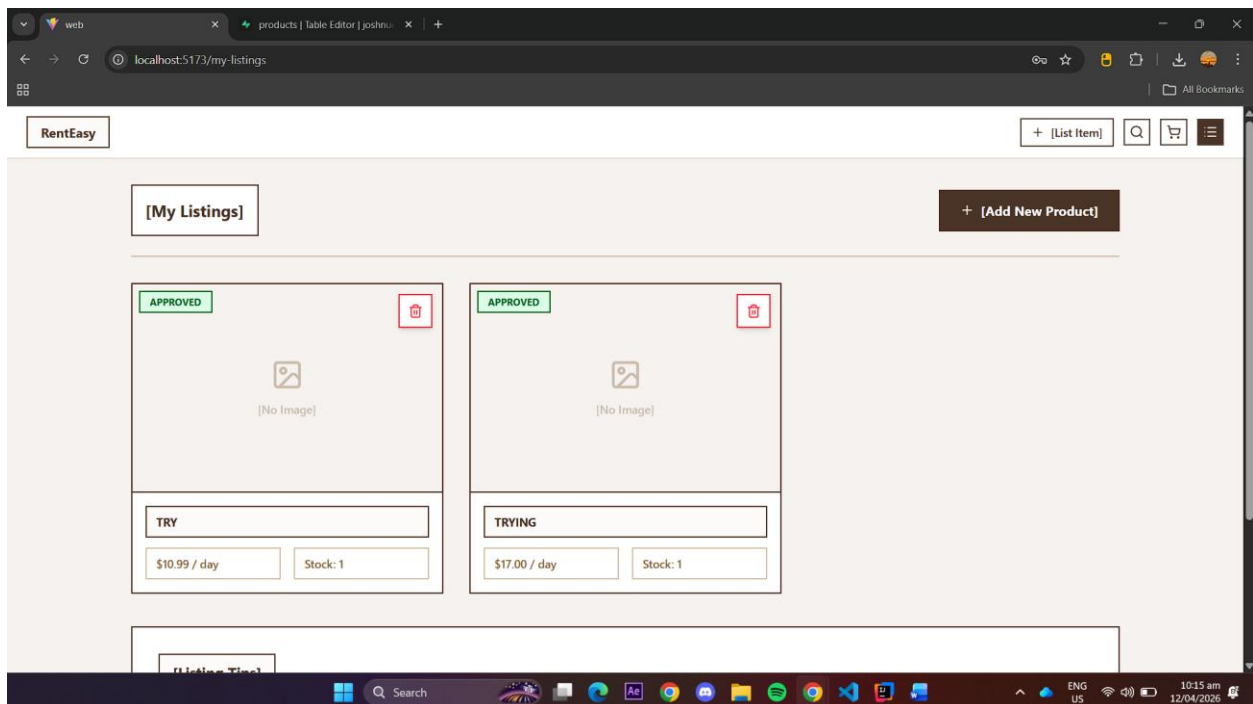
Item Listing Page



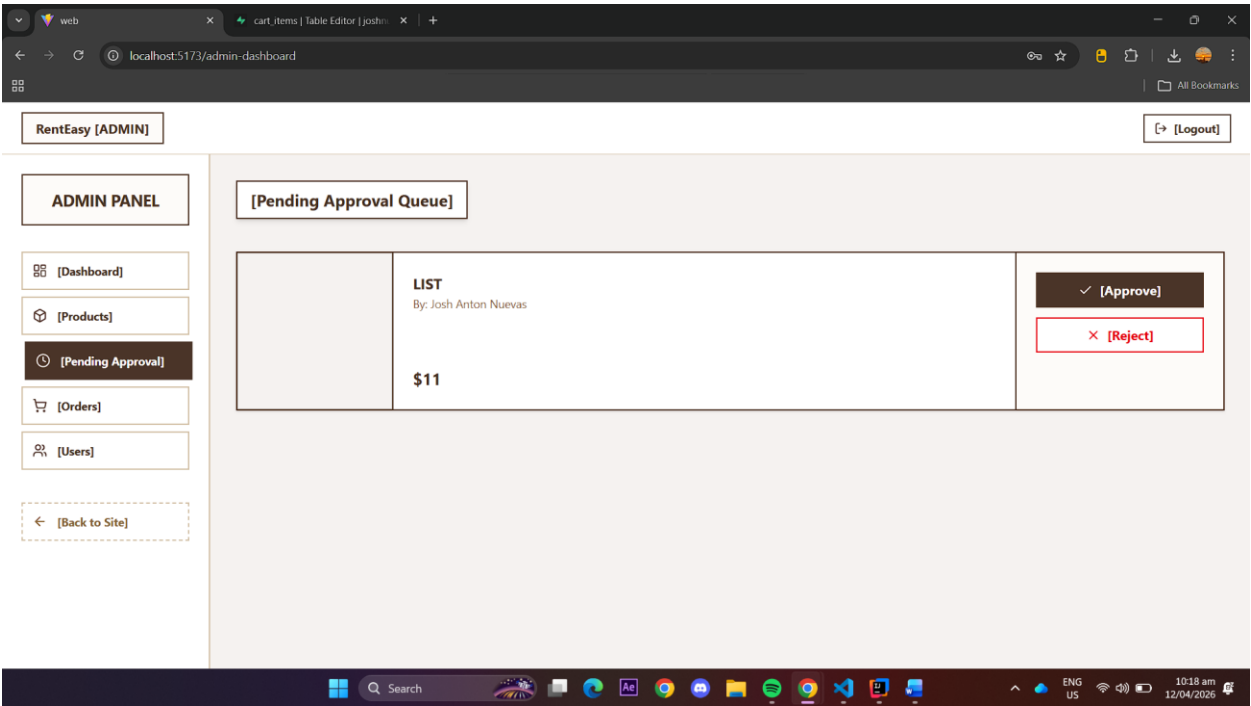
My Cart Page



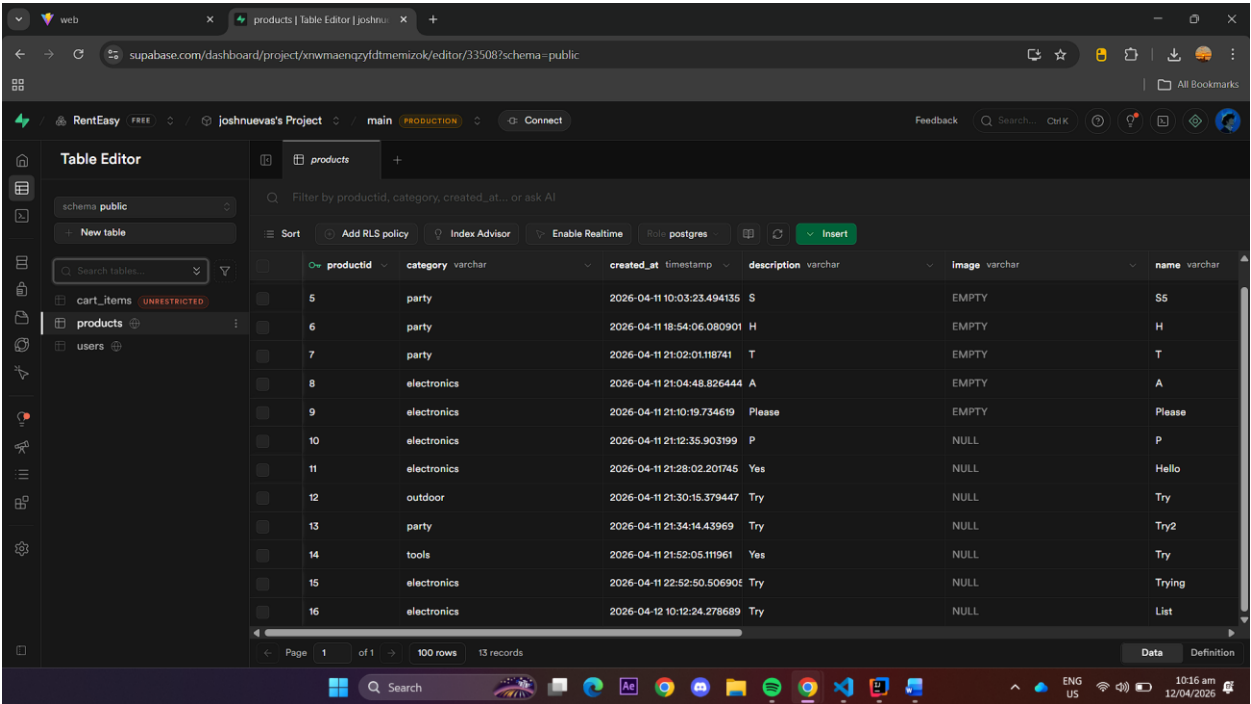
My Listings Page

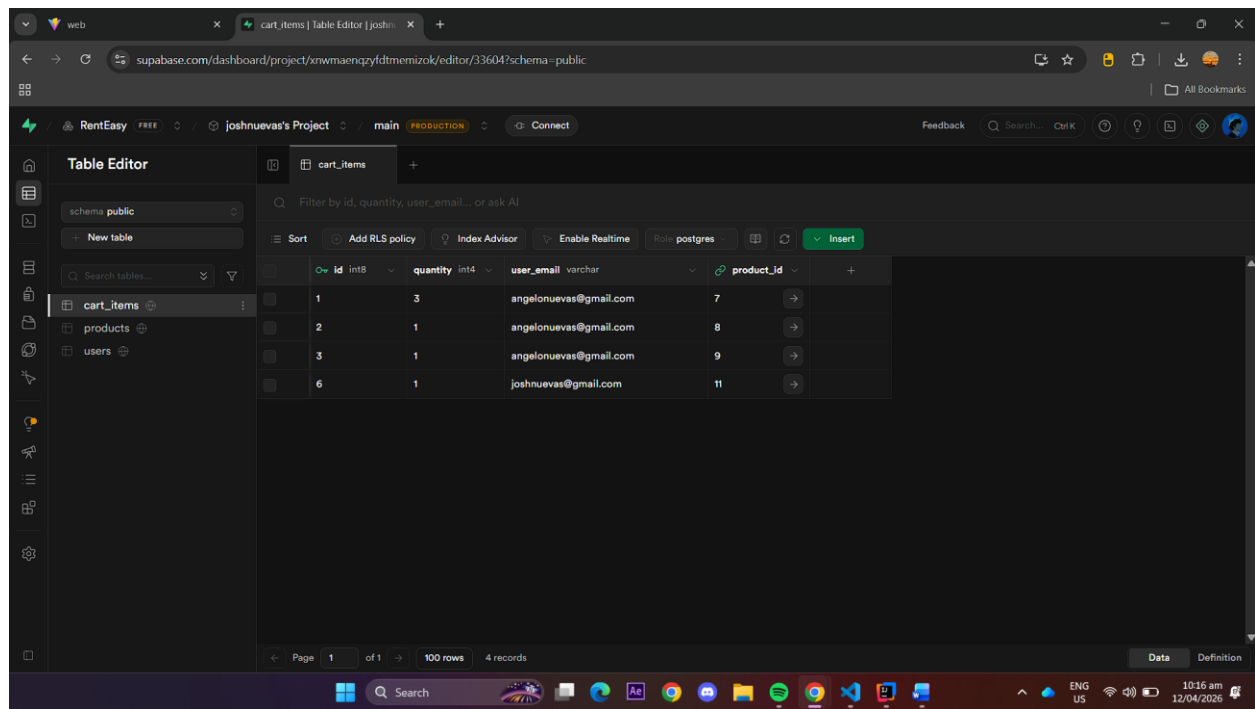


Pending Approval Page



Database Record of Products and Cart Items





Final Commit: <https://github.com/joshnuevas/IT342-Nuevas-RentEasy/commit/33ebc47911adbdd26971c9b26fbeb2238e2e143>

How Product Listing & Browsing Works The browsing experience is designed to allow users to securely view available rental items and add them to their cart, while utilizing dynamic logic to prevent invalid actions.

- **Data Retrieval & Filtering:** The frontend fetches the marketplace inventory using the GET `/api/products/all-` approved endpoint. This ensures that users only see items that have been successfully vetted and approved by an administrator.
- **Dynamic UI State:** While products load, the system simultaneously fetches the user's active cart data. It cross-references productIds to instantly update the UI—automatically disabling buttons and labeling them as "[In Cart]" for previously selected items, or "[Your Listing]" to prevent users from renting their own gear.
- **Secure Cart Addition:** When an item is selected, a secure POST request is dispatched to the backend. It includes the product ID, the user's email, and the JWT Bearer token to ensure the action is authenticated and safely bound to the correct user.

How the Shopping Cart System Works The cart system provides persistent, secure session management, ensuring users retain their selected rental items across different logins and devices.

- **Persistent Database Storage:** Unlike simple local-storage carts, items are saved directly to the backend database. The Cart Service seamlessly handles the logic to either insert a new CartItem record or increment the quantity if the item already exists in the user's cart.
- **Real-time Modification:** The cart interface allows users to increase/decrease quantities (PUT `/api/cart/{id}/quantity`) or remove items entirely (DELETE `/api/cart/{id}`). These actions immediately update the database and trigger a frontend recalculation of the total order summary.
- **Authorization Gate:** All cart endpoints are strictly protected. The backend checks the JWT token on every request to ensure users can absolutely only view, modify, or delete items within their own specific cart.

How User Listing Management Works The "My Listings" dashboard empowers users to act as vendors, giving them full control over the items they have submitted to the platform.

- **Contextual Data Fetching:** The system securely fetches the user's specific inventory using GET `/api/products`. It validates the user's JWT token and filters the resulting payload dynamically, so users only see products tied to their currentUserEmail.
- **Lifecycle Status Tracking:** Users can track the administrative status of their listings via visual UI tags, instantly seeing if a product is currently PENDING review, actively APPROVED for the marketplace, or REJECTED.
- **Inventory Control:** Users maintain complete control over their submissions. They can remove an item from the platform at any time using the secured DELETE `/api/products/{id}` endpoint, which safely executes the removal from the database.

API, Security & Database Integration The platform's features are powered by a robust Spring Boot backend, protected by custom security filters and connected to a relational database.

- **Stateless Authentication:** Every protected request is intercepted by the JwtAuthenticationFilter. This filter extracts the Bearer token, validates its cryptographic signature via JwtUtil, and establishes the security context without relying on traditional server sessions.
- **CORS & Route Protection:** The SecurityConfig strictly defines Cross-Origin Resource Sharing (CORS) policies to allow the React frontend (localhost:5173) to communicate safely with the Java backend. It explicitly locks down sensitive endpoints (like Cart management and Product deletion) while leaving read-only endpoints open to the public.
- **Relational Data Mapping:** Data is managed using Spring Data JPA. The system handles complex relationships between core entities: Product (containing price, status, and owner details) and CartItem (mapping authenticated users to specific products and quantities), ensuring strict referential integrity across the rental marketplace.